



.equals?

.equals

- One of the most fundamental cornerstones of the design of the Java standard library is that there is a single concept of equality.
- This really helped with early Java adoption.
- But on the long term, it is also creating many issues.

Equals is hard

- Equals and hashCode are used a lot in collections.
- The first (wrong) guess would be that a 'Cat' class should have an equals method taking another 'Cat' as follows:

```
public boolean equals(Cat other){ /*..*/ } //BAD
```

- This is not the case: the correct signature is instead

```
public boolean equals(Object other){ /*..*/ } //GOOD
```

- We will see why in the next few slides

Equals in collections

- Consider the following types:

```
class Object{}  
class Animal extends Object{}  
class Cat extends Animal{}  
class SiameseCat extends Cat{}
```

- We have SiameseCat < Cat < Animal < Object.

What happens when we make lists/sets of those types?

```
List<Object>      os  
List<Animal>     as  
List<Cat>         cs  
List<SiameseCat> ss
```

- The list of animals can contain cats too.

So, we could call `as.contains(aSiameseCar)`,
but also `as.contains(anAnimal)`.

What should happen then?

```
boolean equals(Cat other){ /*...*/ } //BAD
boolean equals(Object other){ /*...*/ } //GOOD
```

```
class Object{}
class Animal extends Object{}
class Cat extends Animal{}
class SiameseCat extends Cat{}
```

```
List<Object>      os
List<Animal>      as
List<Cat>         cs
List<SiameseCat>  ss
```

The list `as` can contain Animals, Cats are Animals, so `as` can contain Cats. Let say 'bob' the Cat is in `as`.

If `Cat.equals` took a `Cat`, could we call `as.contains(myAnimal)`?

How to propagate contains to 'bob'?

`bob.equals(myAnimal)` would be ill typed!

We can not call a method wanting a Cat and pass just an Animal.

```
boolean equals(Cat other){ /*...*/ } //BAD
boolean equals(Object other){ /*...*/ } //GOOD
```

```
class Object{}
class Animal extends Object{}
class Cat extends Animal{}
class SiameseCat extends Cat{}
```

```
List<Object>      os
List<Animal>      as
List<Cat>         cs
List<SiameseCat>  ss
```

The list `as` can contain Animals, Cats are Animals, so `as` can contain Cats. Let say 'bob' the Cat is in `as`.

If `Cat.equals` took a `Cat`, could we call `as.contains(mySiameseCat)`?

May be? The siamese cat is also a cat so we could pass it.

But... how can the collection know that 'bob' has a `Cat.equals` method taking a `Cat`?

- The list only knows about the Animal type!

```
boolean equals(Cat other){ /*...*/ }    //BAD
boolean equals(Object other){ /*...*/ }  //GOOD
```

```
class Object{}
class Animal extends Object{}
class Cat extends Animal{}
class SiameseCat extends Cat{}
```

```
List<Object>    os
List<Animal>    as
List<Cat>       cs
List<SiameseCat> ss
```

So, a List of Animal must be able to call at least `.equals(Animal)`

- So, Animal would need at least a `.equals(Animal)`
- Since Cat extends Animal, they would need `.equal(Animal)` too.
- But what about the list of Objects?
a List of Object must be able to call at least `.equals(Object)`
- So Animals and Cats should also have a `.equals(Object)`

```
boolean equals(Cat other){ /*...*/ }    //BAD
boolean equals(Object other){ /*...*/ }  //GOOD
```

```
class Object{}
class Animal extends Object{}
class Cat extends Animal{}
class SiameseCat extends Cat{}
```

```
List<Object>      os
List<Animal>      as
List<Cat>         cs
List<SiameseCat>  ss
```

.equals(Cat) .equals(Animal) .equals(Object)

- Those would be 3 different methods in overloading (the ugly brother)
 - **BAD:** They could have 3 different and incompatible behaviors
 - **BAD:** We may accidentally call the wrong one
 - **GOOD:** .equals(Object) is general enough to be used in all cases.
- The Java design is to rely ONLY on .equals(Object)

```
boolean equals(Cat other){ /*...*/ }    //BAD
boolean equals(Object other){ /*...*/ }  //GOOD
```

```
class Object{}
class Animal extends Object{}
class Cat extends Animal{}
class SiameseCat extends Cat{}
```

```
List<Object>    os
List<Animal>    as
List<Cat>       cs
List<SiameseCat> ss
```

The Java design is to rely ONLY on .equals(Object)

- So, .equals(Cat) .equals(Animal) and other variants are often sign of errors and misconceptions.

How to implement `.equals(Object)`?

- Option 1(The BEST):
 - Do not override `.equals` and `.hashCode` manually, but use records whenever possible. Records make a good predefined job of implementing `equals` and `hashCode`.
- Option 2:
 - Accept that implementing `.equals` is much harder than what your intuition would suggest and learn the following slides carefully.
 - It is going to be a surprising and painful journey...

Animal and Cat .equals

```
class Animal extends Object{ ...  
    public boolean equals(Object obj) {
```

```
    }  
    String name;  
    int age;  
    Animal(String name, int age){ this.name= name; this.age= age; }  
}  
class Cat extends Animal{ ...  
    public boolean equals(Object obj) {
```

```
    }  
    double cuteness;  
    Cat(String name,int age,double cuteness){ super(name,age); this.cuteness= cuteness;}  
}
```

Animal with
name and age

Cat also have
a cuteness



Animal and Cat .equals

```
class Animal extends Object{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance
        if (obj == null){ return false; }
        if (getClass() != obj.getClass()){ return false; }
        Animal other= (Animal) obj;
        return age == other.age && Objects.equals(name, other.name);
    }
    String name;
    int age;
    Animal(String name, int age){ this.name= name; this.age= age; }
}

class Cat extends Animal{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance, omitted in later examples
        if (!super.equals(obj)){ return false; }
        if (getClass() != obj.getClass()){ return false; }
        Cat other= (Cat) obj;
        return Double.doubleToLongBits(cuteness)==Double.doubleToLongBits(other.cuteness);
    }
    double cuteness;
    Cat(String name,int age,double cuteness){ super(name,age); this.cuteness= cuteness;}
}
```



This is how eclipse would implement them

Animal and Cat .equals

```
class Animal extends Object{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance

    }
    String name;
    int age;
    Animal(String name, int age){ this.name= name; this.age= age; }
}
class Cat extends Animal{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance, omitted in later examples

    }
    double cuteness;
    Cat(String name,int age,double cuteness){ super(name,age); this.cuteness= cuteness;}
}
```

Line by line: the first check
is just for performance



Animal and Cat .equals

```
class Animal extends Object{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance
        if (obj == null){ return false; }

    }
    String name;
    int age;
    Animal(String name, int age){ this.name= name; this.age= age; }
}
class Cat extends Animal{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance, omitted in later examples

    }
    double cuteness;
    Cat(String name,int age,double cuteness){ super(name,age); this.cuteness= cuteness;}
}
```

If the other is null, it must not be equal to us

The fact that we are in one of our methods means that we are not null!



Animal and Cat .equals

```
class Animal extends Object{ ...  
    public boolean equals(Object obj) {  
        if (this == obj){ return true; } //just for performance  
        if (obj == null){ return false; }  
    }
```

In the derived class, we check for super.equals instead.

```
String name;  
int age;  
Animal(String name, int age){ this.name= name; this.age= age; }  
}
```

super.equals is also checking that obj is not null.

```
class Cat extends Animal{ ...  
    public boolean equals(Object obj) {  
        if (this == obj){ return true; } //just for performance, omitted in later examples  
        if (!super.equals(obj)){ return false; }  
    }
```

```
double cuteness;  
Cat(String name, int age, double cuteness){ super(name, age); this.cuteness= cuteness; }  
}
```



Animal and Cat .equals

```
class Animal extends Object{ ...  
    public boolean equals(Object obj) {  
        if (this == obj){ return true; }//just for performance  
        if (!super.equals(obj)){ return false; }//bad variant
```

```
    }  
    String name;  
    int age;  
    Animal(String name, int age){ this.name= name; this.age= age; }  
}
```

```
class Cat extends Animal{ ...  
    public boolean equals(Object obj) {  
        if (this == obj){ return true; }//just for performance, omitted in later examples  
        if (!super.equals(obj)){ return false; }
```

```
    }  
    double cuteness;  
    Cat(String name,int age,double cuteness){ super(name,age); this.cuteness= cuteness;}  
}
```

What if had super
also in Animal?

Object.equals would
return false if obj is null.
Does it means that it
works but is less efficient?

Nope!, Object.equals would
only be true if the are
pointer equals, so this
variant would fail too often.



Animal and Cat .equals

```
class Animal extends Object{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance
        if (obj == null){ return false; }
        if (getClass() != obj.getClass()){ return false; }
        Animal other= (Animal) obj;

    }
    String name;
    int age;
    Animal(String name, int age){ this.name= name; this.age= age; }
}
class Cat extends Animal{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance, omitted in later examples
        if (!super.equals(obj)){ return false; }
        if (getClass() != obj.getClass()){ return false; }
        Cat other= (Cat) obj;

    }
    double cuteness;
    Cat(String name,int age,double cuteness){ super(name,age); this.cuteness= cuteness;}
}
```

Then we check that
obj class is identical
to our class.

Note that getClass is
dynamically dispatched.
So if 'this' is
dynamically a Cat

'this.getClass()'
would be
'Cat.class'



Animal and Cat .equals

```
class Animal extends Object{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance
        if (obj == null){ return false; }
        if (Animal.class != obj.getClass()){ return false; }
        Animal other= (Animal) obj;

    }
    String name;
    int age;
    Animal(String name, int age){ this.name= name; this.age= age; }
}
class Cat extends Animal{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance, omitted in later examples
        if (!super.equals(obj)){ return false; }
        if (getClass() != obj.getClass()){ return false; }
        Cat other= (Cat) obj;

    }
    double cuteness;
    Cat(String name,int age,double cuteness){ super(name,age); this.cuteness= cuteness;}
}
```

Then we check that
obj class is identical
to our class.

Note that getClass is
dynamically dispatched.
So if 'this' is
dynamically a Cat

'this.getClass()'
would be
'Cat.class'

So, hardcoding
Animal.class
would not work!



Animal and Cat .equals

```
class Animal extends Object{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; } //just for performance
        if (obj == null){ return false; }
        if (getClass() != obj.getClass()){ return false; }
        Animal other= (Animal) obj;
        return age == other.age && Objects.equals(name, other.name);
    }
    String name;
    int age;
    Animal(String name, int age){ this.name= name; this.age= age; }
}

class Cat extends Animal{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; } //just for performance, omitted in later examples
        if (!super.equals(obj)){ return false; }
        if (getClass() != obj.getClass()){ return false; }
        Cat other= (Cat) obj;
        return Objects.equals(cuteness, other.cuteness); //Would this work?
    }
    double cuteness;
    Cat(String name, int age, double cuteness){ super(name, age); this.cuteness= cuteness; }
}
```

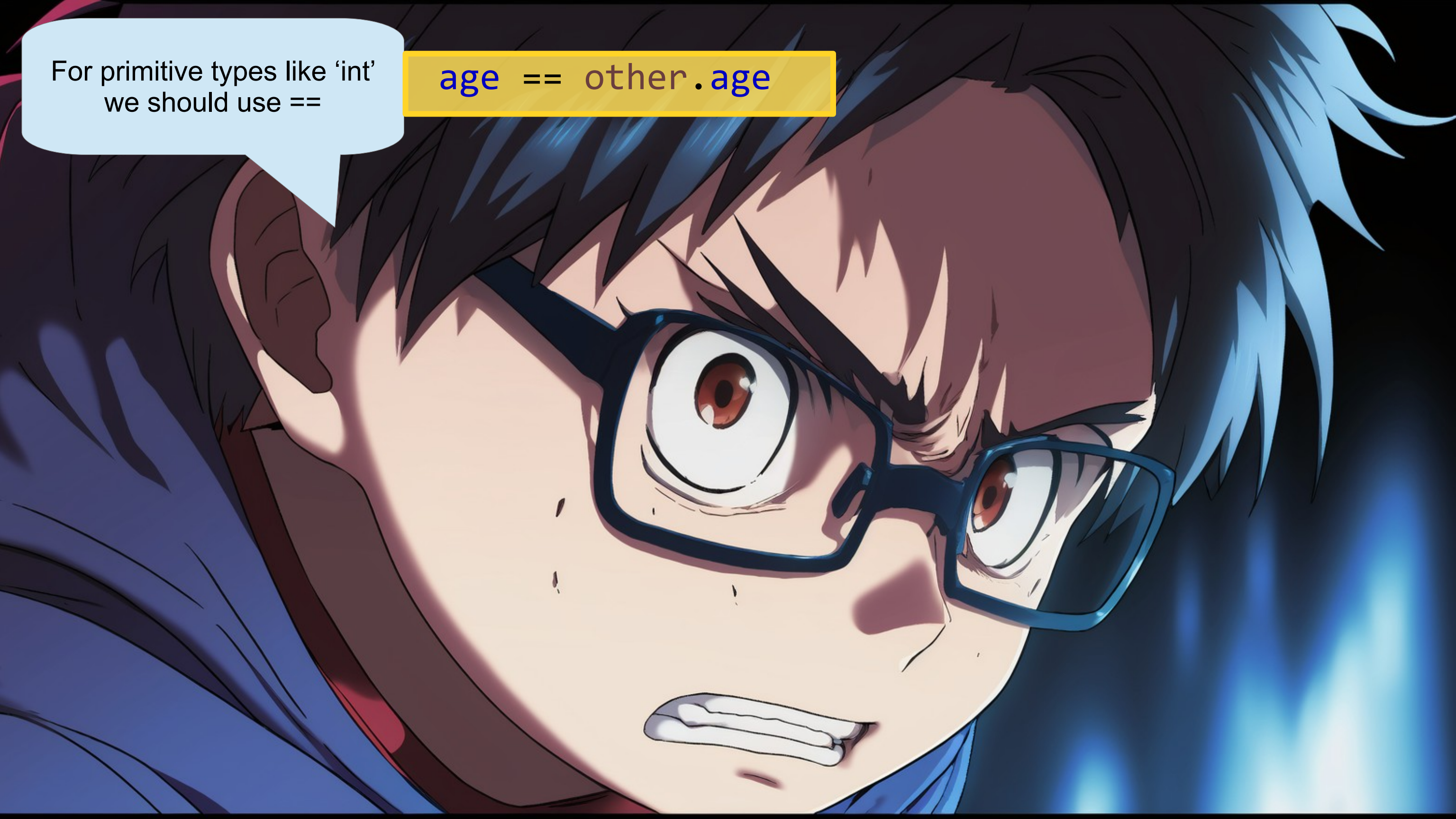
Finally we check that all the fields are equals.

This is also not trivial. We have many different special cases



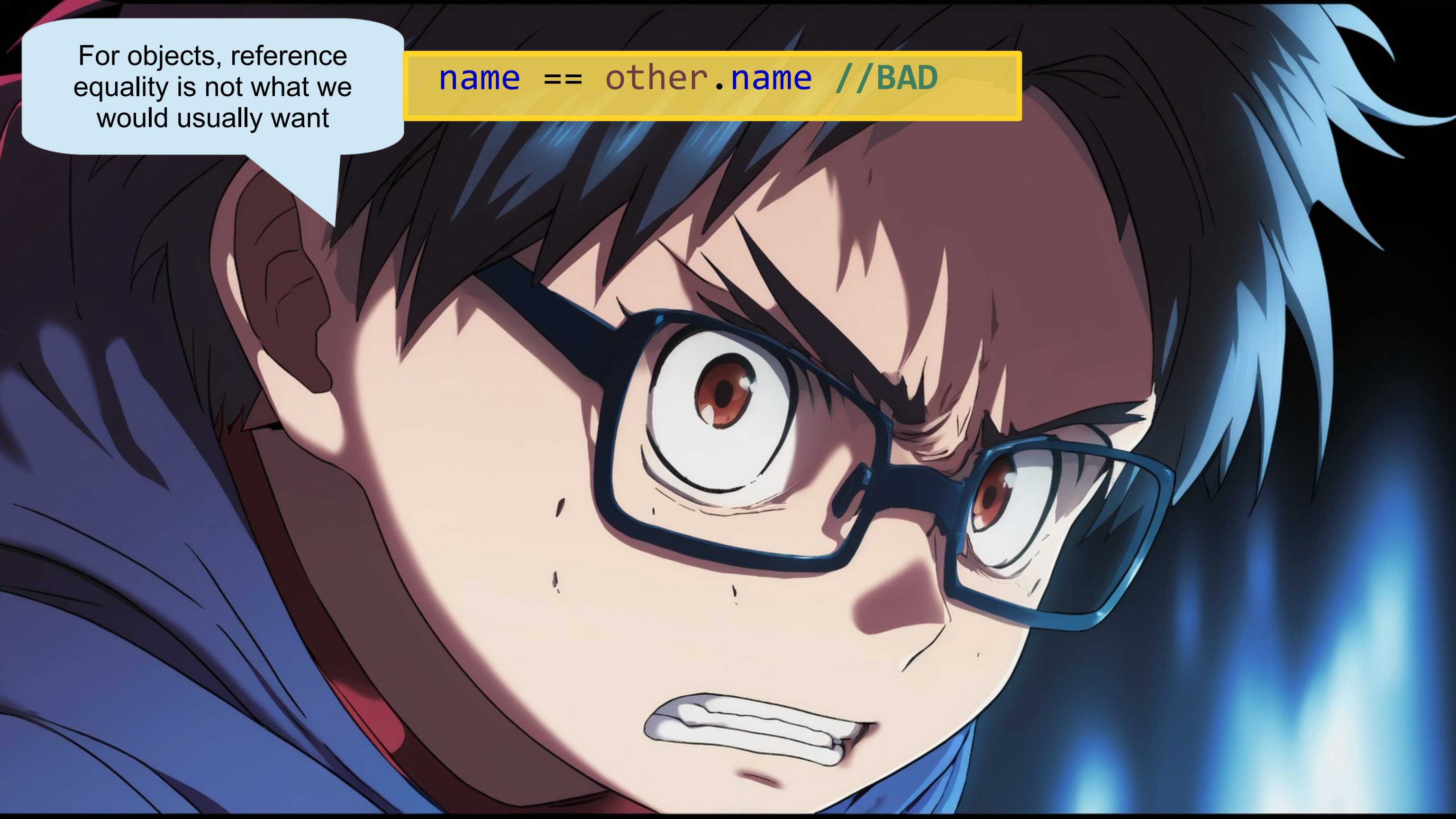
For primitive types like 'int'
we should use ==

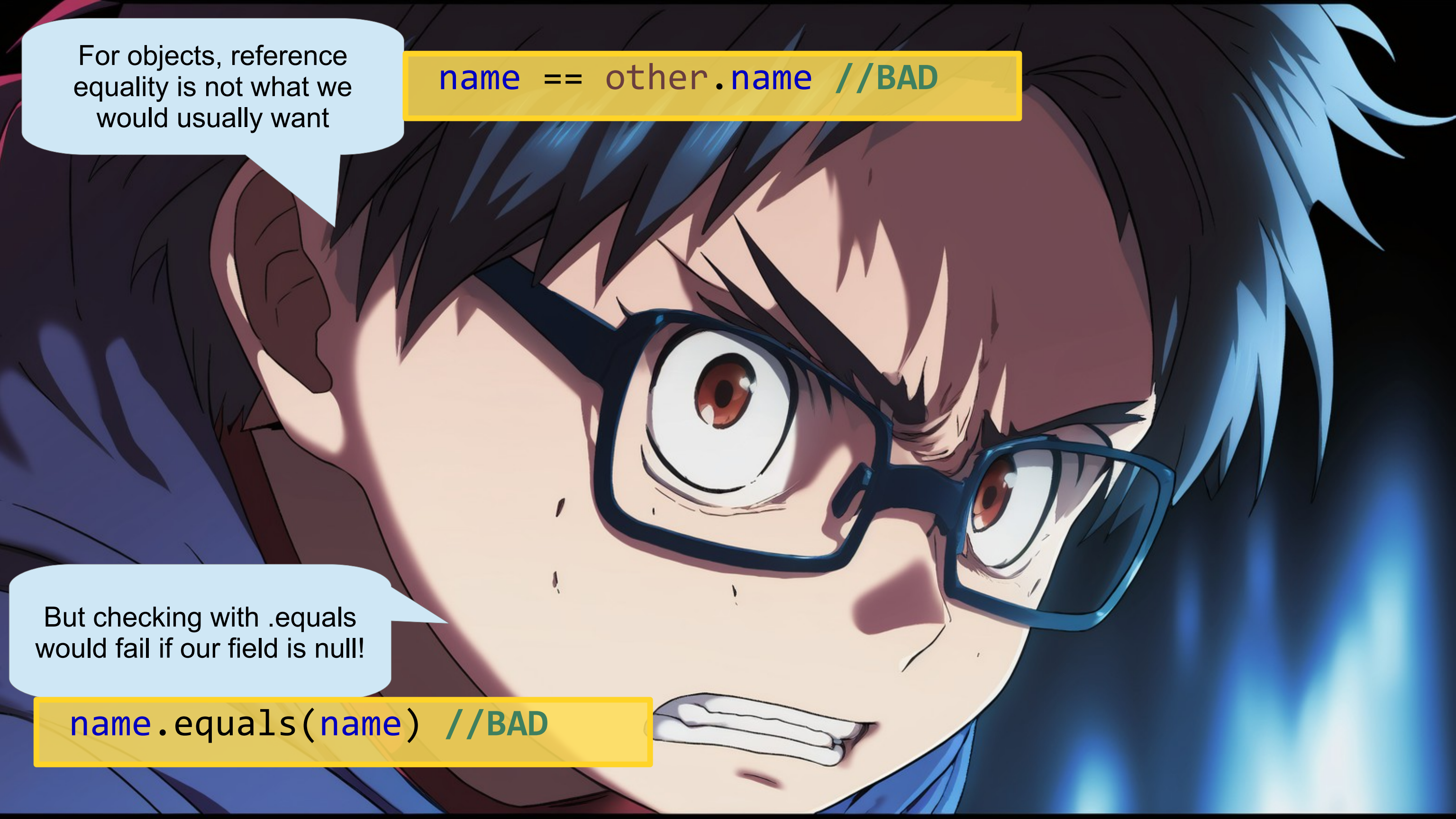
```
age == other.age
```



For objects, reference equality is not what we would usually want

```
name == other.name //BAD
```



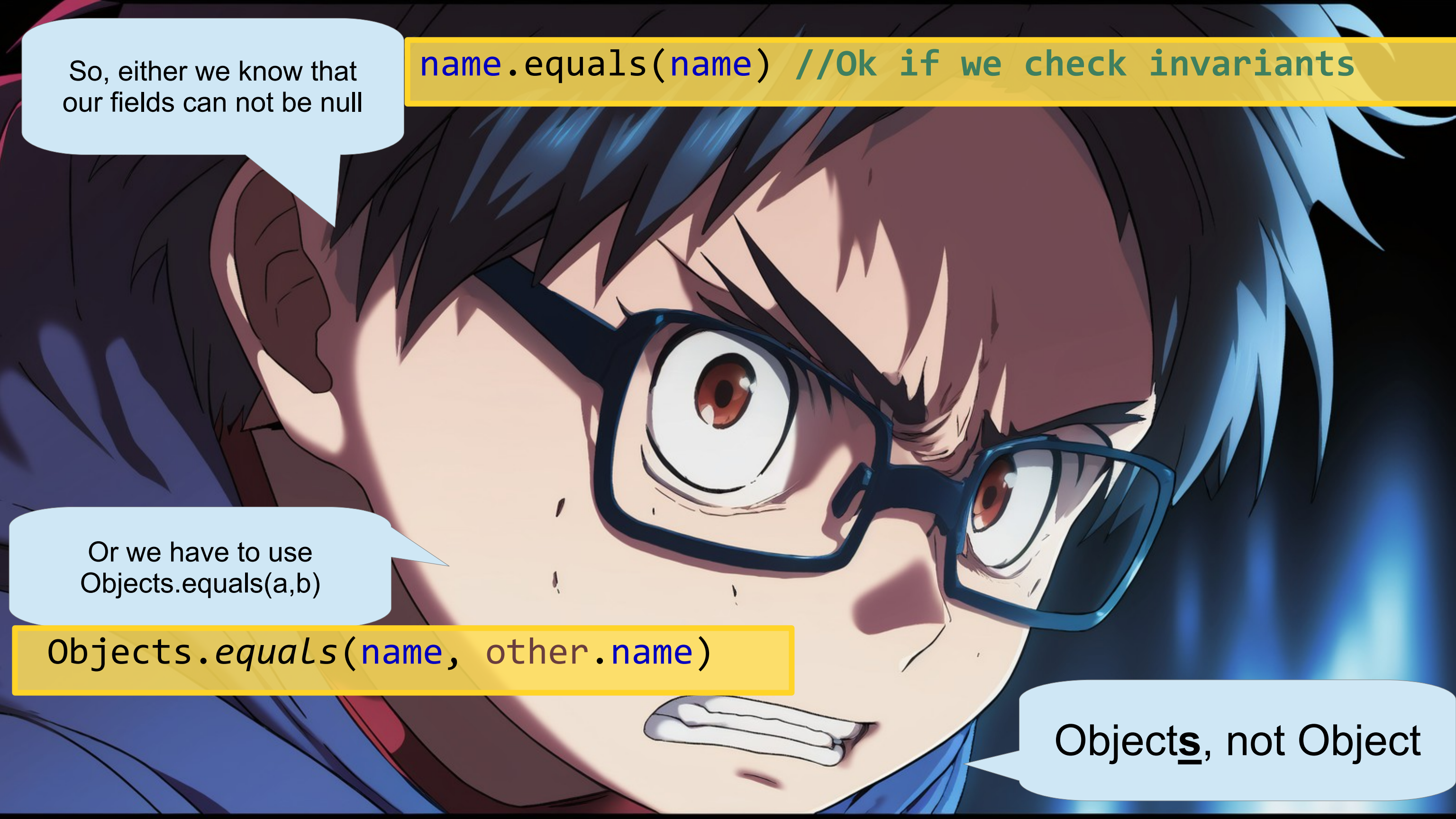


For objects, reference equality is not what we would usually want

```
name == other.name //BAD
```

But checking with `.equals` would fail if our field is null!

```
name.equals(name) //BAD
```

A close-up anime-style illustration of a young man with spiky brown hair and large black-rimmed glasses. He has a surprised or intense expression, with wide eyes and a slightly open mouth showing his teeth. The background is dark with some blue and purple light effects.

So, either we know that
our fields can not be null

```
name.equals(name) //Ok if we check invariants
```

Or we have to use
`Objects.equals(a,b)`

```
Objects.equals(name, other.name)
```

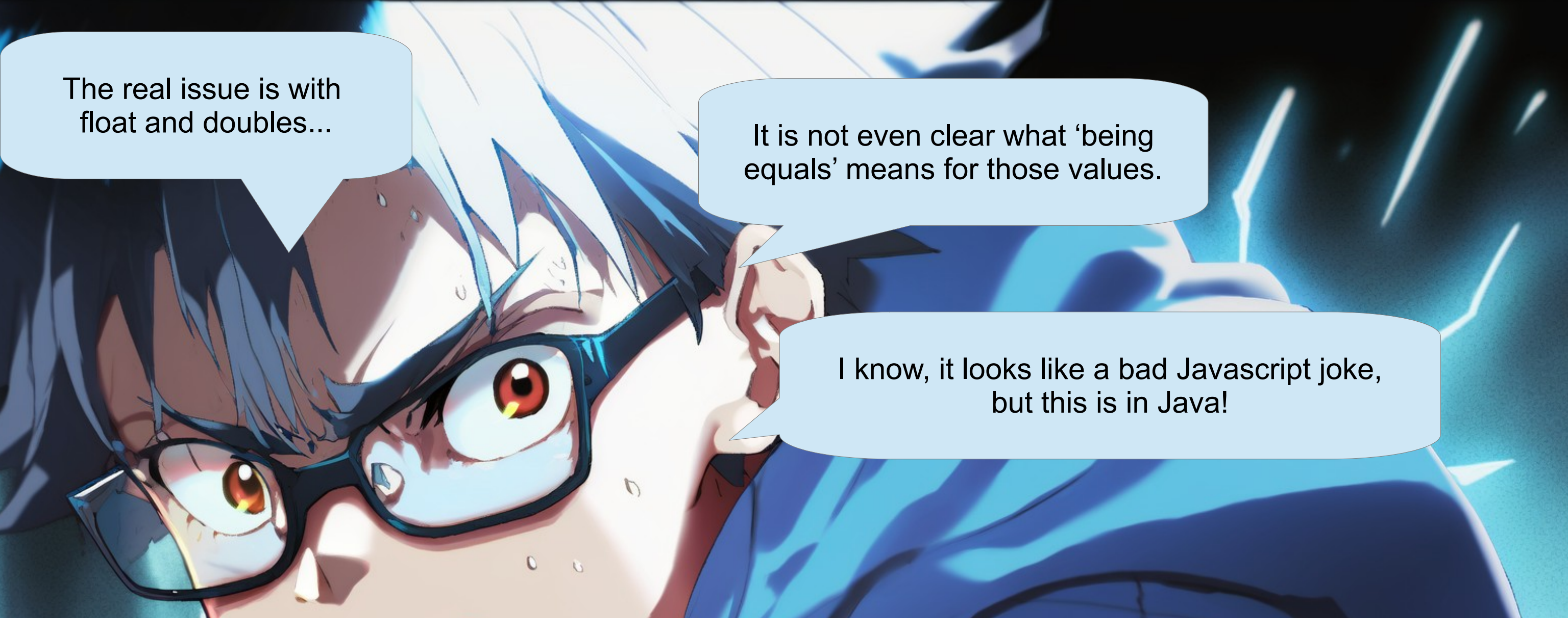
`Objects`, not `Object`



There are more
special cases!

Elements of 'enum' can be
compared with `==`.

The language guarantees that
there is only one instance for
each constant.



The real issue is with
float and doubles...

It is not even clear what 'being
equals' means for those values.

I know, it looks like a bad Javascript joke,
but this is in Java!

```
-0.0f    ==    0.0f;           //true
new Float(-0.0f).equals(new Float(0.0f));    //false
new Float(Float.NaN).equals(new Float(Float.NaN)); //true
Float.NaN == Float.NaN;       //false
Float.floatToIntBits(Float.NaN) == Float.floatToIntBits(Float.NaN) //true
```



```
Float.floatToRawIntBits(weight) == Float.floatToRawIntBits(other.weight)
```

If you want to check if two floats are actually identical

floatToRawIntBits is the best way

Or, maybe you do not care how a NaN was produced? Use floatToIntBits

```
Float.floatToIntBits(weight) == Float.floatToIntBits(other.weight)
```



Same for doubles, just use

`Double.doubleToRawLongBits`
or `Double.doubleToLongBits`

Or better, just avoid
floats and doubles
like the plague!

Person and Student .equals

```
class Animal extends Object{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance
        if (obj == null){ return false; }
        if (getClass() != obj.getClass()){ return false; }
        Animal other= (Animal) obj;
        return age == other.age && Objects.equals(name, other.name);
    }
    String name;
    int age;
    Animal(String name, int age){ this.name= name; this.age= age; }
}

class Cat extends Animal{ ...
    public boolean equals(Object obj) {
        if (this == obj){ return true; }//just for performance, omitted in later examples
        if (!super.equals(obj)){ return false; }
        if (getClass() != obj.getClass()){ return false; }
        Cat other= (Cat) obj;
        return Double.doubleToLongBits(cuteness) == Double.doubleToLongBits(other.cuteness);
    }
    double cuteness;
    Cat(String name,int age,double cuteness){ super(name,age); this.cuteness= cuteness;}
}
```



So, this is how eclipse would implement them

BAD Person and Student .equals

```
class Animal extends Object{ ...
    public boolean equals(Object obj) {
        ➡ if (!(obj instanceof Animal a)){ return false; }
        return age == a.age && Objects.equals(name, a.name);
    }
    String name;
    int age;
    Animal(String name, int age){ this.name= name; this.age= age; }
}
class Cat extends Animal{ ...
    public boolean equals(Object obj) {
        if (!super.equals(obj)){ return false; }
        if (!(obj instanceof Cat c)){ return false; }
        ➡ return Double.doubleToLongBits(cuteness) == Double.doubleToLongBits(c.cuteness);
    }
    double cuteness;
    Cat(String name, int age, double cuteness){ super(name, age); this.cuteness= cuteness; }
}
```

But surely with the new Java features we could be better, for example

In this way we can even remove the null check since 'instanceof' is always false for 'null'

Can you write a counter example?

A case where this new implementation would make a major mess?



How to search for a counter example?

- What are the cases?
 - Two equal animals?
 - Two different animals?
 - Two equal cats?
 - Two different cats?

Go by cases!



How to search for a counter example?

- What are the cases?
 - Two equal animals?
 - Two different animals?
 - Two equal cats?
 - Two different cats?
- Those all pass, if those are all the cases you are trying, you still have not internalized the nature of OO!
- So, what are the other cases?

Go by cases!



How to search for a counter example?

- What are the cases?
 - Two equal animals?
 - Two different animals?
 - Two equal cats?
 - Two different cats?
- Those all pass, if those are all the cases you are trying, you still have not internalized the nature of OO!
- So, what are the other cases?
 - Cat with Animal
 - Animal with Cat
- But.. they would simply all fail, right?
How can an Animal be equal to a Cat?

Go by cases!



How to search for a counter example?

- What are the cases?
 - Two equal animals?
 - Two different animals?
 - Two equal cats?
 - Two different cats?
- Those all pass, if those are all the cases you are trying, you still have not internalized the nature of OO!
- So, what are the other cases?
 - Cat with Animal
 - Animal with Cat
- But.. they would simply all fail, right?
How can an Animal be equal to a Cat?

Go by cases!

That would be a bug,
and a bug is exactly
what we are searching
for!



How to search for a counter example?

```
new Cat("Bob",4,300d)  
    .equals(new Animal("Bob",4));
```

```
new Animal("Bob",4)  
    .equals(new Cat("Bob",4,300d));
```

- We would really like for those two to return the same result
- We would like for those two to be both false
- Is it?

Cat and Animal

Animal and Cat



How to search for a counter example?

```
new Cat("Bob",4,300d)
    .equals(new Animal("Bob",4));
```

```
Cat.equals(Object obj) {
    if (!super.equals(obj)){ return false; } //Pass
    if (!(obj instanceof Cat c)){ return false; } //Fail
    return ...;
}
```

Cat and Animal

- Ok for the first case
 - Note: the super call passes since the shared fields are the same.



How to search for a counter example?

```
new Animal("Bob",4)
    .equals(new Cat("Bob",4,300d));
```

```
Animal.equals(Object obj) {
    if (!(obj instanceof Animal a)){ return false; } //Pass
    return age == p.age && Objects.equals(name, p.name); //Pass
}
```

- But the second is passing... an animal can be equal to a cat if the fields of animal just so happen to be equals...

so...

!! a.equals(b) but not b.equals(a) !!

Animal and Cat

We discussed this already: any way to hardcode the dynamic type of the 'other' object does not work.





Oh No!

Nothing is ever simple and intuitive. The more a concept looks obvious the hardest it is to tame it!



And this is just the start.
I'm not even discussing
how to handle `.equals` for
circular object graphs

There is not even
agreement about what
being structurally equal
even means in that setting!

.equals: formal properties

Everything is better
with more math!



- It is reflexive: for any non-null reference value x , $x.equals(x)$ should return true.
- It is symmetric: for any non-null reference values x and y , $x.equals(y)$ should return true if and only if $y.equals(x)$ returns true.
- It is transitive: for any non-null reference values x , y , and z , if $x.equals(y)$ returns true and $y.equals(z)$ returns true, then $x.equals(z)$ must return true.
- It is consistent: for any non-null reference values x and y , multiple invocations of $x.equals(y)$ consistently return true or consistently return false, provided no information used in the equals comparisons on the objects is modified.
- For any non-null reference value x , $x.equals(null)$ should return false.

.hashCode: formal properties

Everything is better
with more math!



- When invoked on the same object more than once during an execution of a Java application, the hashCode method must consistently return the same integer, provided no information used in equals comparisons on the object is altered. The result need not remain consistent from one execution of an application to another execution of the same application.
- If two objects are equal according to the equals(Object) method, then calling the hashCode method on any of the two objects must produce the same integer result.
- Note: if two objects are not equal according to method equals(Object), it is not required that calling the hashCode method on each of the two objects must produce distinct integer results. However, producing distinct integer results for unequal objects may improve the performance of hash tables.



So, a correct
implementation of
hashCode can just return
0 all the times!

Those objects will simply
be slow when used in
HashMaps and HahsSets

Usually, all fields are used
to compute the hashCode,
but...

In some cases, excluding
some fields from hashCode
would be a good idea and
speed them up!



Next:
Generics!